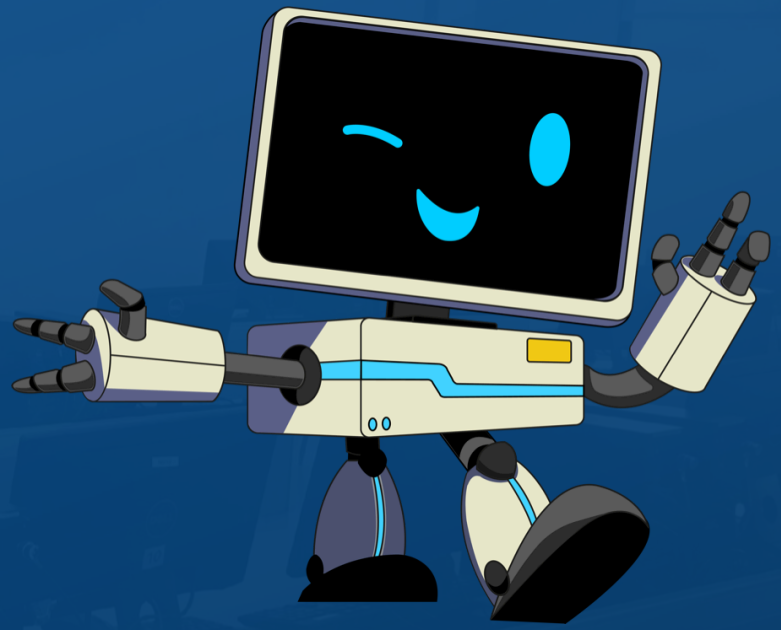
UIFCE
UNIDAD DE INFORMÁTICA



Curso:
Introducción a Python y R

Sesión 1:
¿Qué es programar? Primeros pasos con Python

INTRODUCCIÓN A PYTHON Y R · SESIÓN 1

¿Qué es programar?

Nombre del monitor(a): Yony Sebastian Chaparro Mesa

Estudiante Auxiliar

Correo: uniic_bog@unal.edu.co

Tel: 3165000 ***Ext:*** 12301

Facultad de Ciencias Económicas
Unidad de Informática
Sede Bogotá

UNIVERSIDAD NACIONAL DE COLOMBIA

0

Presentación del curso

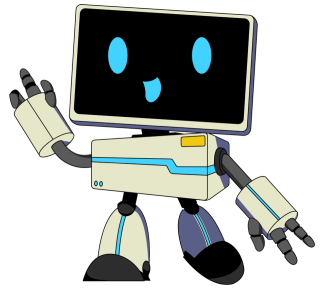
Información y reglas del curso libre

Correo: uniic_bog@unal.edu.co
Teléfono: 3165000 **ext** 12301

UIFCE
UNIDAD DE INFORMÁTICA

Información de interés

Cursos Libres UIFCE · 2026-2



FECHAS

26 oct – 28 nov

de 2026

HORARIO

Martes y jueves

2:00 – 4:00 p. m.

LUGAR

Sala 6

Sala de Informática 6, Facultad de Ciencias Económicas

¿CUÁNTAS SESIONES?

10 sesiones

20 horas en total

ASISTENCIA MÍNIMA

8 sesiones

80 % del curso

MATERIAL DEL CURSO

Moodle FCE

Notebooks, ejercicios y entregas

Correo: uniic_bog@unal.edu.co

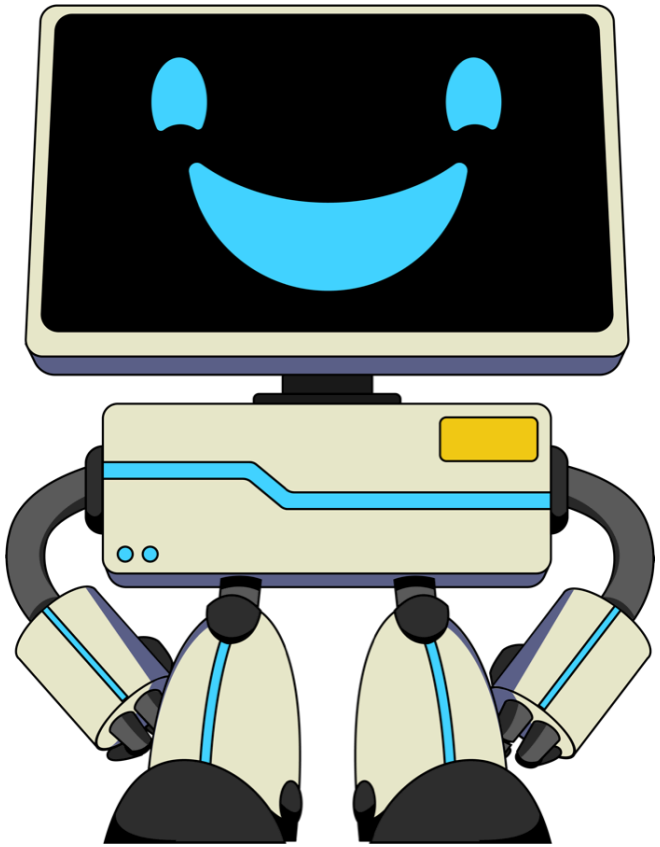
Teléfono: 3165000 **ext** 12301

UIFCE
UNIDAD DE INFORMÁTICA

Objetivo del curso

> Aprender la **programación básica** en los lenguajes **Python y R** para que pueda ser utilizada en las actividades cotidianas.

Sin experiencia previa **Desde el navegador** **Con datos reales**



Módulos del curso

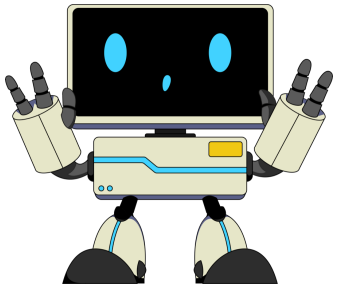
1	¿Qué es programar? Primeros pasos	PYTHON
2	Condicionales y lógica	PYTHON
3	Listas, bucles y funciones	PYTHON
4	Primeros datos reales: pandas	PYTHON
5	El puente a R	R
6	Manipulación de datos con dplyr	R
7	Visualización con ggplot2	R
8	Python y R lado a lado · grupos de proyecto	AMBOS
9	Taller de proyecto	AMBOS
10	Presentaciones y cierre	AMBOS

Correo: uniic_bog@unal.edu.co
Teléfono: 3165000 **ext** 12301

UIFCE
UNIDAD DE INFORMÁTICA

Reglas de los cursos libres

Cursos Libres UIFCE



¿Qué pasa si no asisto a la primera clase?

Pierde el cupo

¿Qué plazo tengo para cancelar?

Las tres primeras sesiones

¿Qué pasa si pierdo el curso por fallas?

Sanción durante 6 meses

¿Qué pasa si se presentan casos de plagio?

Pierde el curso y sanción por 6 meses

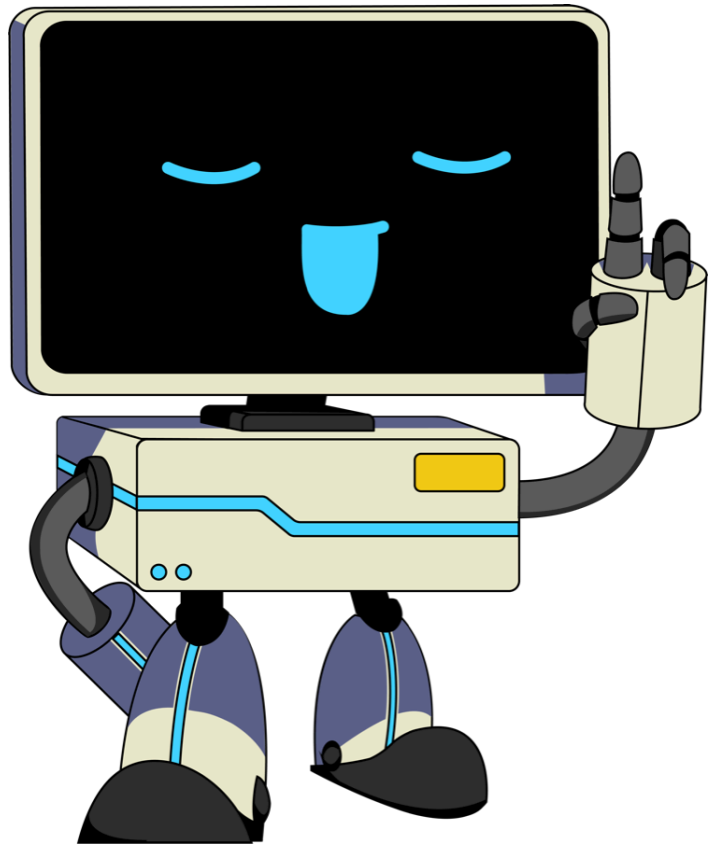
Correo: uniic_bog@unal.edu.co

Teléfono: 3165000 **ext** 12301

UIFCE
UNIDAD DE INFORMÁTICA

Durante las clases

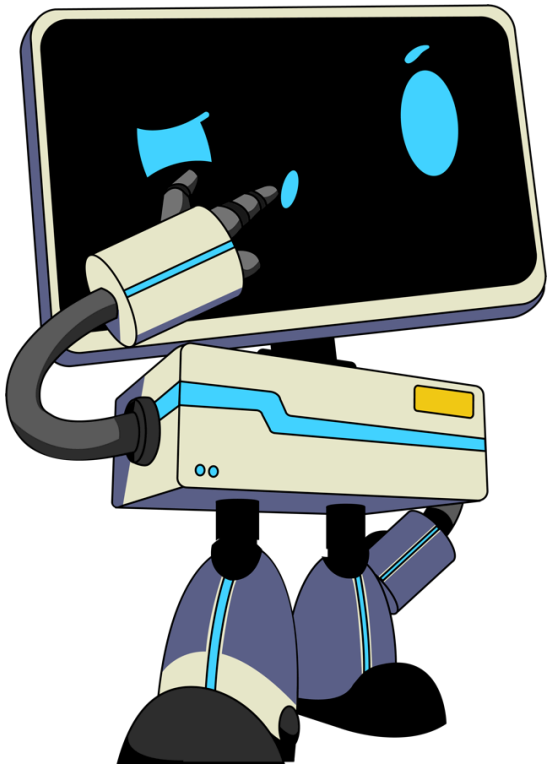
- La clase empieza a las **2:00 p. m.** Lleguen puntuales.
- Se toma lista con un **formulario durante la clase**.
- No se reciben entregas **después de la fecha máxima** ni **por correo electrónico**.
- Todas las entregas se hacen en el **Moodle FCE**.



Criterios de evaluación

CRITERIO	PONDERACIÓN
Ejercicios cortos semanales	40 %
Proyecto final	50 %
Participación y retroalimentación entre pares	10 %

- El curso se aprueba con **3,5**.
- Se darán **bonos** durante las clases.



Certificado y reclamos



¿Cómo obtengo el certificado?

Obtener una nota igual o superior a **3,5**.

Responder la **encuesta de evaluación y retroalimentación**.



¿Y si no estoy de acuerdo con mi nota?

Tiene **una semana** después de terminado el curso para presentar el reclamo a uniic_bog@unal.edu.co.

REDES DE LA UIFCE

Web: fce.unal.edu.co/uifce **Facebook:** facebook.com/uifce **X (Twitter):** @FCEuifce **YouTube:** Unidad de Informática y Comunicaciones FCE

Contenido

Sesión 1 · Python en Google Colab

1. ¿Qué es programar?
2. Google Colab
3. Variables y tipos de datos
4. Operaciones
5. Leer errores sin miedo
6. ¡A practicar!

1

¿Qué es programar?

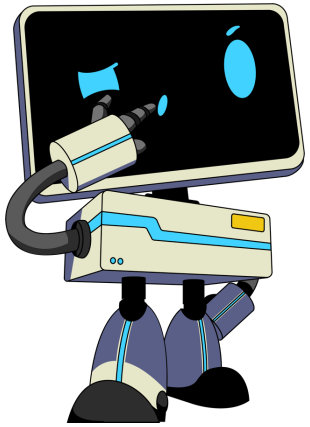
De las instrucciones cotidianas al código

Correo: uniic_bog@unal.edu.co
Teléfono: 3165000 **ext** 12301

UIFCE
UNIDAD DE INFORMÁTICA

Programamos sin saberlo

¿Cuántas veces hoy le diste instrucciones a una máquina?



Una alarma

“Todos los días hábiles a las 6:00 a. m.”

Una receta

Pasos en orden, con cantidades exactas.

Una fórmula

`=PROMEDIO(B2:B30)` en Excel.

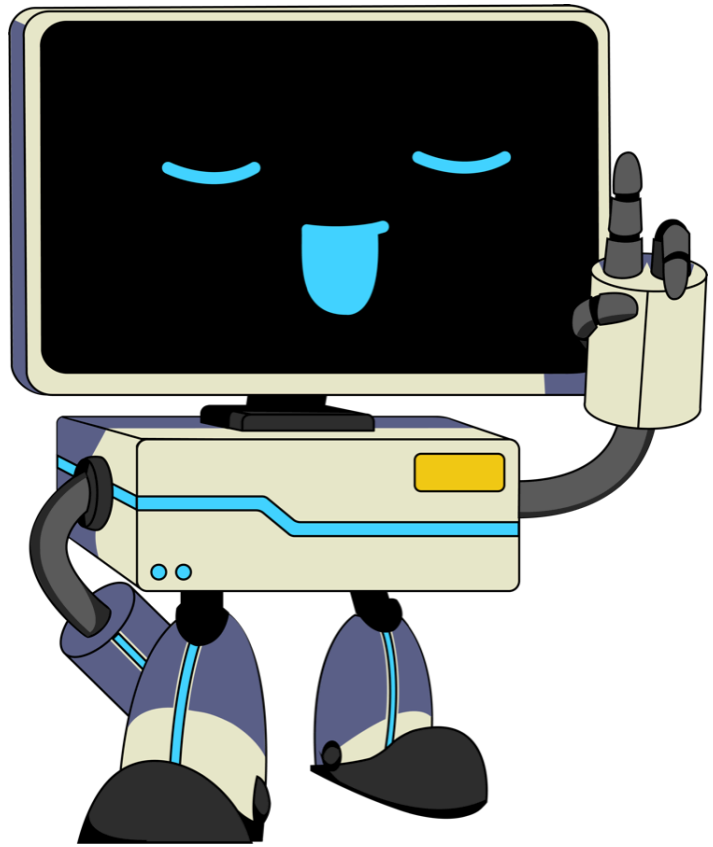
Un filtro

“Si el correo es de la U, muévelo a esta carpeta”.

Programar es **describir, paso a paso, lo que queremos que pase.**

Entonces, ¿qué es programar?

> **Programar** es escribir **instrucciones precisas** que un computador puede ejecutar para **resolver un problema** o hacer una tarea.



Tres palabras clave

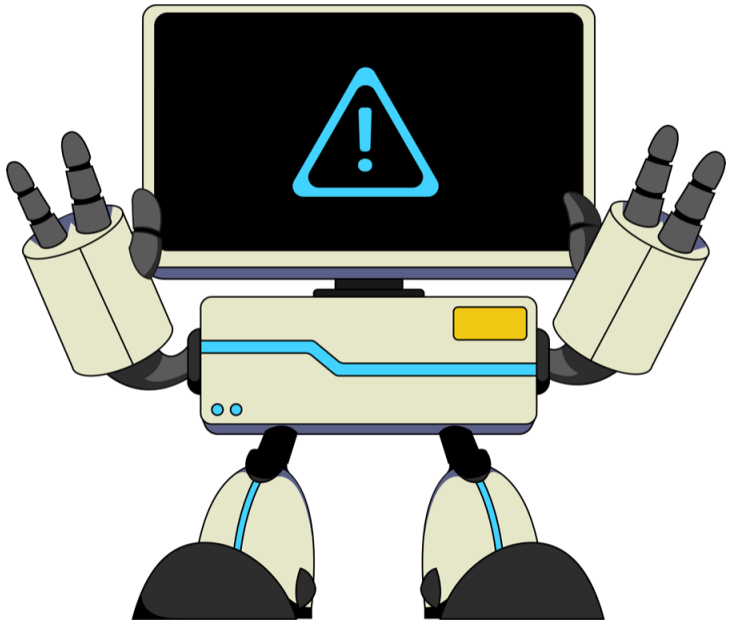
Instrucciones **Precisión** **Problema**

Un **programa** es el resultado: una lista de instrucciones que el computador sigue **en orden**, de arriba hacia abajo.

El computador es literal

Hace exactamente lo que le dices, no lo que quisiste decir.

- No interpreta, no supone, no “entiende el contexto”.
- Si la instrucción está mal escrita, **no la entiende**.
- Una receta para alguien que **no improvisa**.



ACTIVIDAD

La receta al pie de la letra. El grupo le dicta al monitor cómo preparar un sándwich. El monitor obedece *exactamente* lo que escucha. ¿Qué instrucciones faltaron?

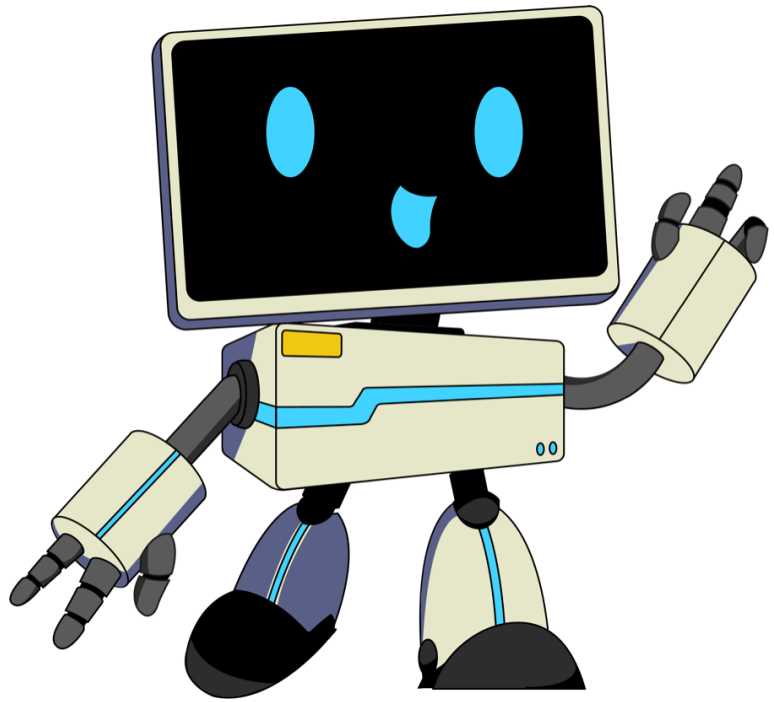
Correo: uniic_bog@unal.edu.co
Teléfono: 3165000 **ext** 12301

Algoritmo: la idea antes del código

Un **algoritmo** es una secuencia **finita y ordenada** de pasos para resolver un problema.

Ejemplo: calcular el promedio de mis notas

1. Reunir todas las notas.
2. Sumarlas.
3. Contar cuántas notas hay.
4. Dividir la suma entre la cantidad.
5. Mostrar el resultado.



Del algoritmo al programa

Problema ¿qué quiero? → **Algoritmo** los pasos → **Código** en un lenguaje → **Ejecución** el computador → **Resultado** 3.5

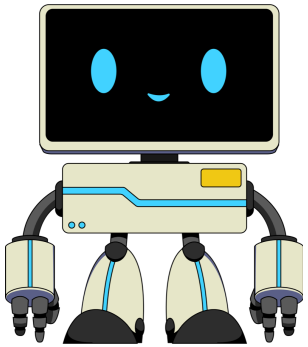
PYTHON

```python
notas = [3.0, 4.0, 3.5]
promedio = sum(notas) / len(notas)
print(promedio)   # 3.5
```

R

```r
notas <- c(3.0, 4.0, 3.5)
promedio <- mean(notas)
print(promedio)   # 3.5
```

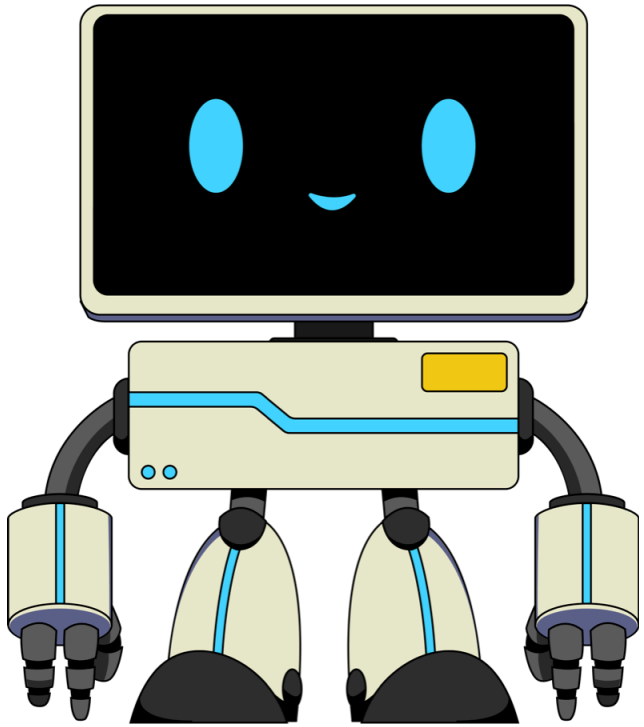
El mismo algoritmo, dos lenguajes. **No hay que entender cada símbolo todavía.**



¿Qué es un lenguaje de programación?

Un idioma intermedio entre las personas y la máquina, con **vocabulario** (palabras reservadas) y **gramática** (sintaxis).

Lenguaje humano	“Calcula el promedio de mis notas”
Alto nivel	`mean(notas)` · Python, R
Bajo nivel	`MOV EAX, [EBX]` · ensamblador
Código máquina	`10110000 01100001`



¿Cómo “entiende” el computador nuestro código?

Compilado

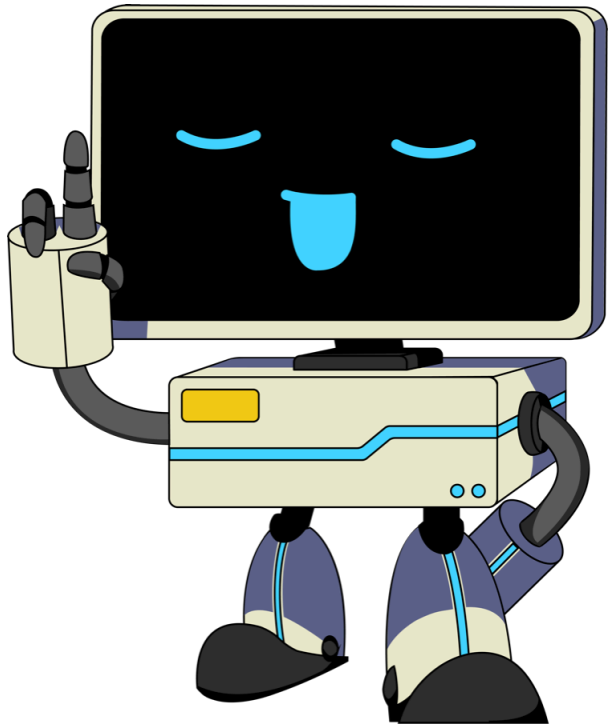
Traduce **todo** el programa antes de ejecutarlo.

Como traducir un libro completo antes de publicarlo.
Ej.: C, Java

Interpretado

Lee y ejecuta **línea por línea**, de arriba hacia abajo.

Como un intérprete en una conferencia.
Ej.: **Python, R**



```
>>> 2 + 3
5
```

Escribes una línea y ves el resultado **al instante**. Ideal para aprender y explorar datos.

Los bloques de cualquier programa

Lo que vamos a ir armando durante el curso

HOY

Datos

Variables y tipos: texto, números, verdadero/falso.

```
edad = 20
nombre = "Ana"
```

HOY

Operaciones

Cálculos y comparaciones.

```
total = 12000 * 3
```

SESIÓN 2

Decisiones

Si pasa A, hacer B.

```
if edad >= 18:
    print("Adulto")
```

SESIÓN 3

Repetición

Hacer algo muchas veces.

```
for n in notas:
    print(n)
```

SESIÓN 3

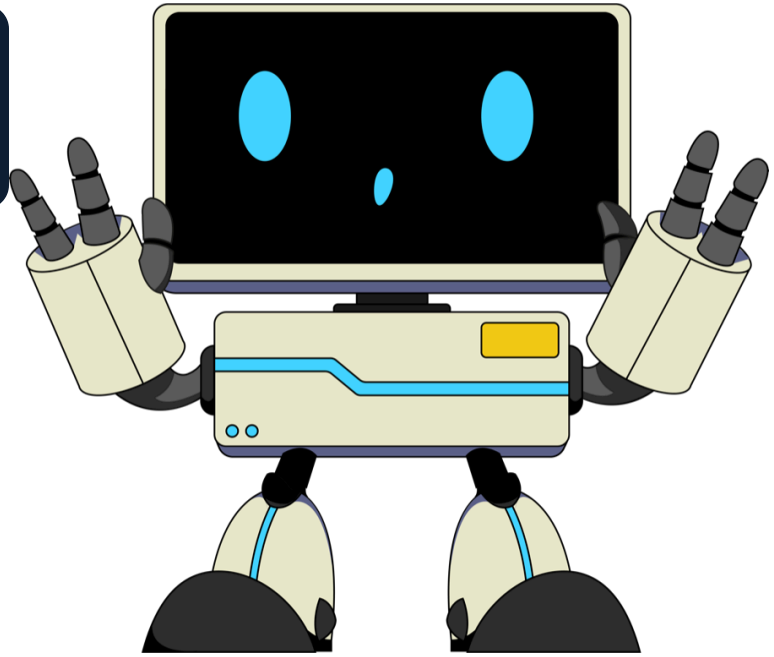
Reutilización

Agrupar pasos con un nombre.

```
def saludar(x):
    print("Hola", x)
```

Equivocarse es parte del proceso

```
print("Hola"
SyntaxError: '(' was never closed
```



- Los errores (*bugs*) son **normales y esperados**.
- Lee la **última línea** del mensaje y el **número de línea**.
- Depurar es parte del trabajo, no una señal de fracaso.

Un error no es un fracaso: es el computador diciéndote dónde mirar.

Correo: uniic_bog@unal.edu.co
Teléfono: 3165000 **ext** 12301

¿Por qué aprender a programar hoy?

Automatizar

Las tareas repetitivas se hacen solas, sin errores de copiar y pegar.

Analizar datos

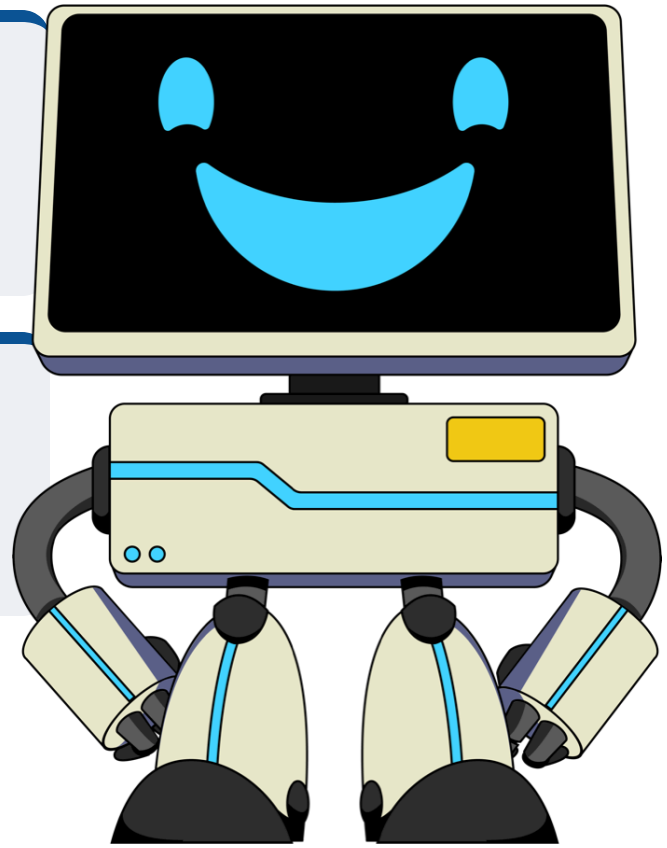
Trabajar con volúmenes que no caben cómodos en una hoja de cálculo.

Reproducibilidad

El mismo código produce el mismo resultado, hoy y en seis meses.

Pensamiento computacional

Descomponer problemas, reconocer patrones y abstraer.



¿Por qué Python y R?

Python

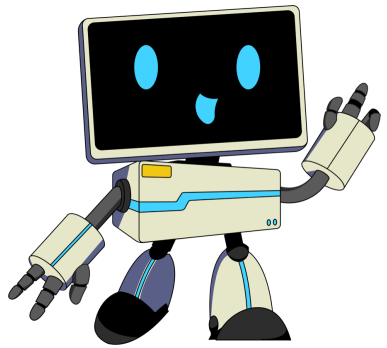
- Propósito general: datos, web, automatización.
- Diseñado para ser legible.
- **Sesiones 1 a 4**, en Google Colab.

R

- Nació para la estadística.
- Muy fuerte en análisis y gráficos.
- **Sesiones 5 a 8**: dplyr, ggplot2 y comparación.

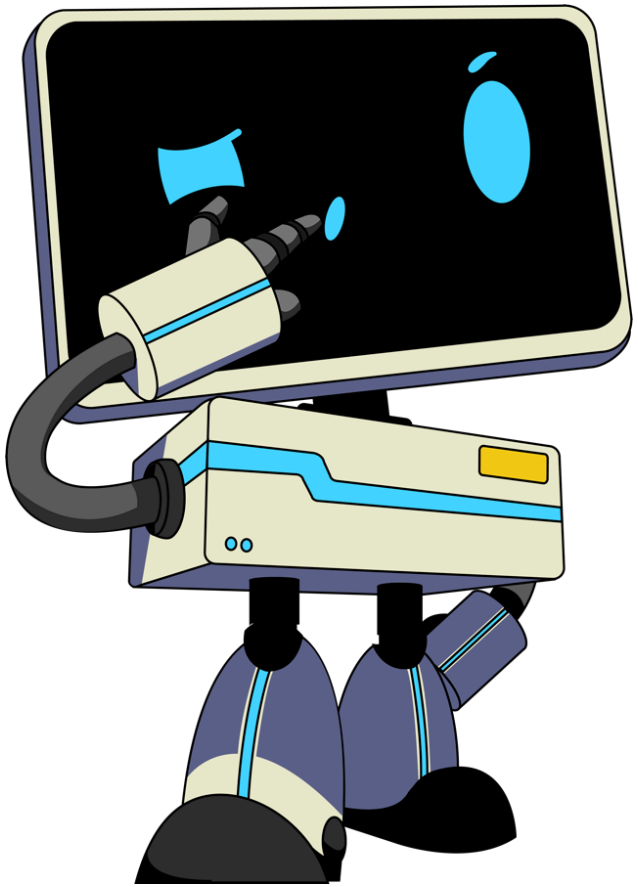
En común: gratuitos, con comunidades enormes y muy usados en análisis de datos.

Ahora veamos cómo se escribe esto... **¡en Colab!**



Para llevar de esta sección

1. Programar es dar **instrucciones precisas**, en orden.
2. Primero el **algoritmo**, después el código.
3. Equivocarse es parte de aprender: **los errores son pistas**.



¿Qué tarea de tu día a día te gustaría automatizar?

2

Google Colab

Nuestro cuaderno de trabajo en la nube

Correo: uniic_bog@unal.edu.co
Teléfono: 3165000 **ext** 12301

UIFCE
UNIDAD DE INFORMÁTICA

¿Qué es un notebook?

Un documento con dos tipos de celdas: texto y código

Sesion01.ipynb + Código + Texto

Mi primer programa

Celda de texto: notas, títulos, explicaciones.

```
print("Hola, mundo")
```

```
Hola, mundo
```

```
2 + 3
```

```
5
```

1. Abre el enlace con tu cuenta de Google.
2. **Archivo → Guardar una copia en Drive.**
3. Escribe en una celda de código.
4. Ejecútala con **Shift** + **Enter**
5. La salida aparece justo debajo.

Hola, mundo

La tradición de todo primer programa

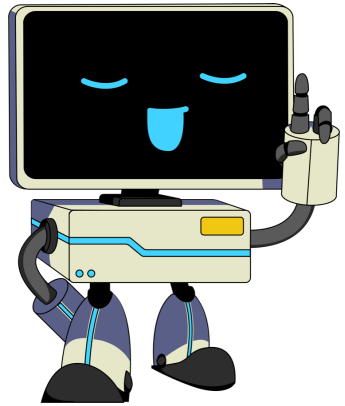
CELDA DE CÓDIGO

```
print("Hola, mundo")
print("Me llamo Ana")
print(2 + 3)
```

SALIDA

```
Hola, mundo
Me llamo Ana
5
```

- `print()` muestra lo que va entre paréntesis.
- El **texto** va entre comillas.
- Los **números** no: Python los calcula.



Práctica en parejas

Con tu pareja de la semana

RETO

En una celda nueva, imprime tu **nombre**, tu **carrera** y tu **frase favorita**, cada uno en su propia línea.

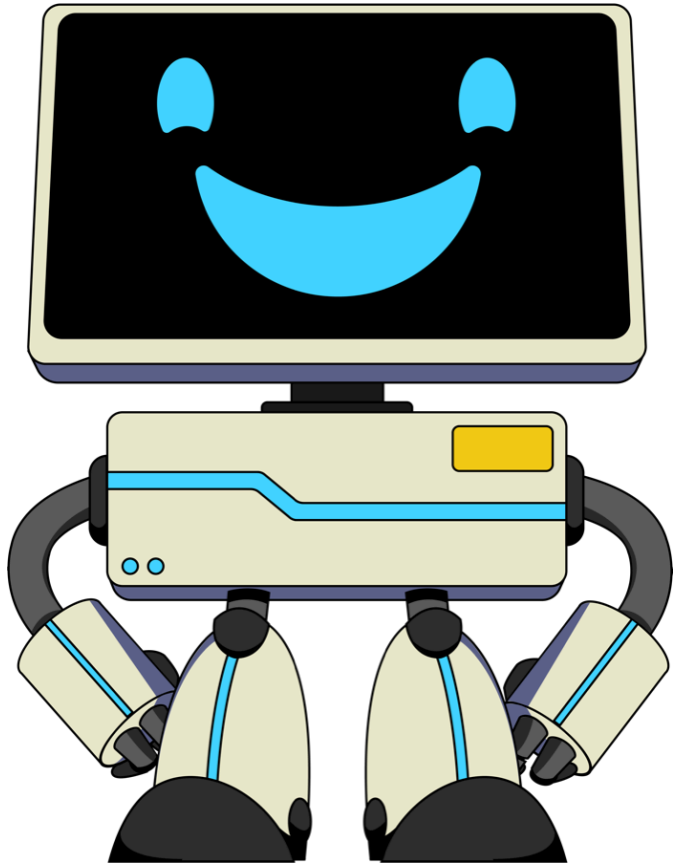
```
print("Ana")
print("Economía")
print("Hecho es mejor que perfecto")
```

Papel verde

¡Terminé! Puedo ayudar.

Papel rojo

Estoy atascado: necesito ayuda.

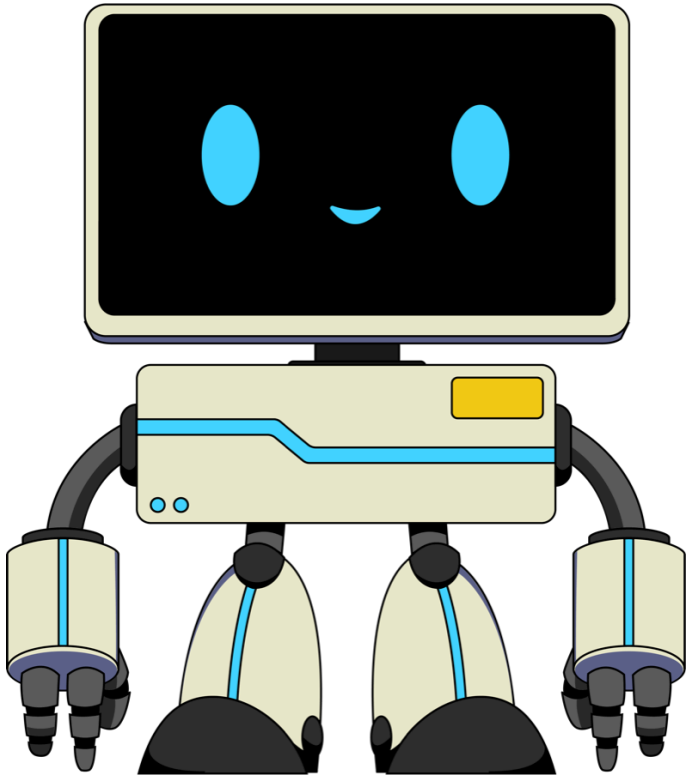


Correo: uniic_bog@unal.edu.co
Teléfono: 3165000 **ext** 12301

Pausa

Estírate y toma agua.

10:00



3

Variables y tipos de datos

Guardar información para usarla después

Correo: uniic_bog@unal.edu.co
Teléfono: 3165000 **ext** 12301

¿Qué es una variable?

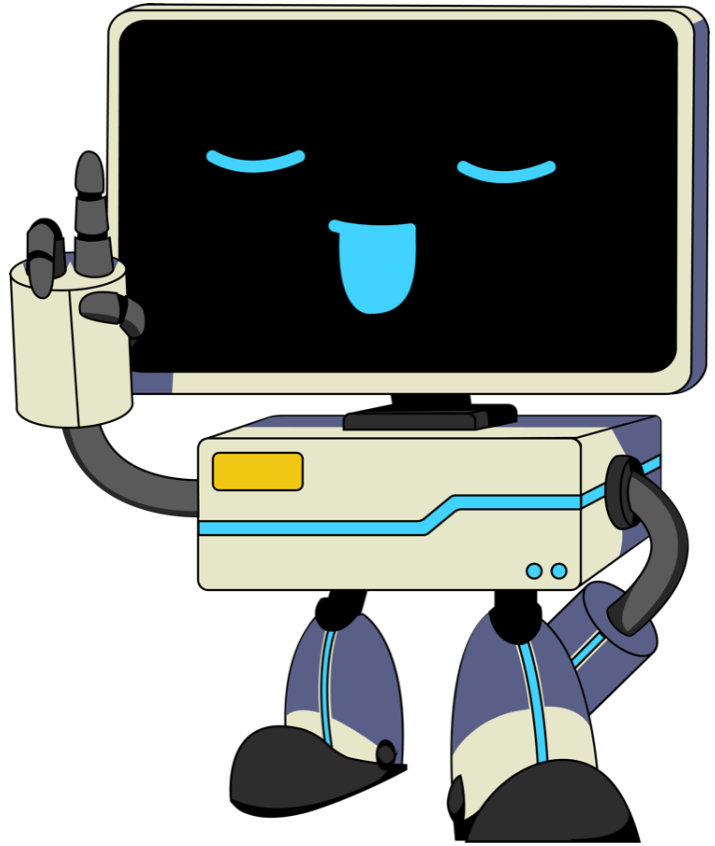
Una **caja con etiqueta**: guarda un dato y lo puedes usar por su nombre.

nombre	edad	estatura
"Ana"	20	1.65

```
edad = 20
```

Se lee: **“guarda 20 en edad”**.

El `=` no es la igualdad de matemáticas.



Tipos de datos básicos

Cada dato tiene un tipo, y el tipo define qué puedes hacer con él

`str`
Texto

Siempre entre comillas.

`"Ana"`

`int`
Entero

Números sin decimales.

`20`

`float`
Decimal

Con punto, no con coma.

`1.65`

`bool`
Verdadero / falso

Con mayúscula inicial.

`True`

```
nombre = "Ana"
edad = 20
print(nombre, "tiene", edad, "años")
print(type(1.65))
```

```
# Salida
Ana tiene 20 años
<class 'float'>
```

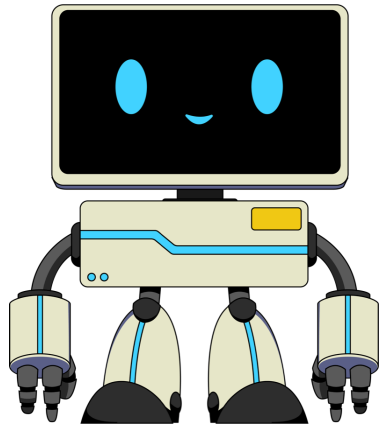
Correo: uniic_bog@unal.edu.co
Teléfono: 3165000 ext 12301

UIFCE
UNIDAD DE INFORMÁTICA

¿Cómo nombrar una variable?

NOMBRE	¿SIRVE?
`edad`	**Sí**
`nota_final`	**Sí** · usa guion bajo
`edad2`	**Sí**
`2edad`	**No** · empieza con número
`nota final`	**No** · tiene espacio
`nota-final`	**No** · Python ve una resta

- Usa nombres que **digan qué guardan**.
- `Edad` y `edad` son **variables distintas**.
- Evita tildes y la ñ en los nombres.



4

Operaciones

El computador como calculadora

Correo: uniic_bog@unal.edu.co
Teléfono: 3165000 **ext** 12301

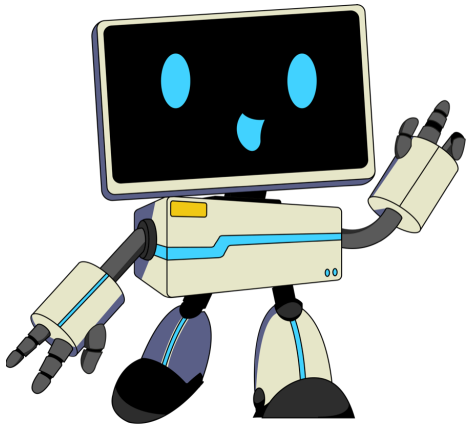
UIFCE
UNIDAD DE INFORMÁTICA

Operadores aritméticos

SÍMBOLO	OPERACIÓN	EJEMPLO	RESULTADO
+	Suma	7 + 2	9
-	Resta	7 - 2	5
*	Multiplicación	7 * 2	14
/	División	7 / 2	3.5
//	División entera	7 // 2	3
%	Residuo	7 % 2	1
**	Potencia	7 ** 2	49

Los cuatro primeros ya los conoces.

Los **resaltados** son los nuevos: `%` sirve, por ejemplo, para saber si un número es par.



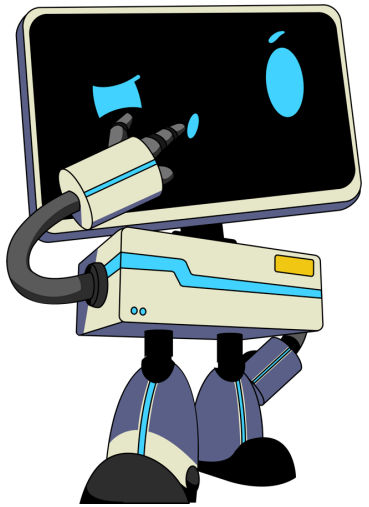
Ejemplo: el total de una compra

```
precio = 12000
cantidad = 3
total = precio * cantidad
print("Total a pagar:", total)
```

```
# Salida
Total a pagar: 36000
```

- Las variables se combinan en cálculos y el resultado se guarda en otra variable.
- Si cambias `precio` y vuelves a ejecutar, el total se actualiza solo.

¿Y si ahora hay que sumar el IVA del 19 %?



5

Leer errores sin miedo

El computador te está diciendo dónde mirar

Correo: uniic_bog@unal.edu.co
Teléfono: 3165000 **ext** 12301

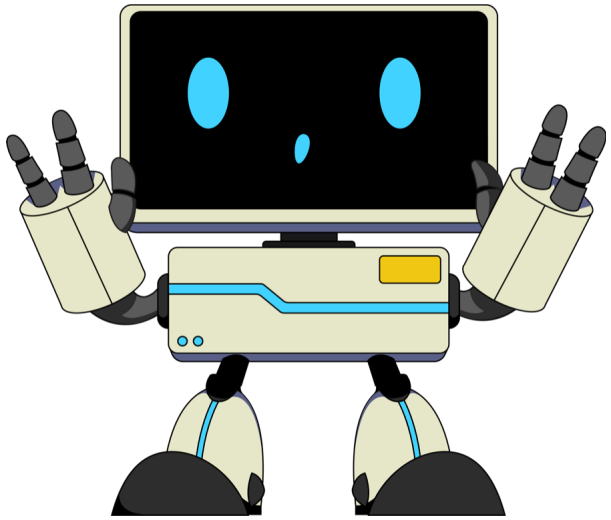
Anatomía de un mensaje de error

```
NameError       Traceback (most recent call last) 2
----> 1 print(Hola) 1

NameError: name 'Hola' is not defined 3
```

Empieza siempre por la última línea: ahí está la pista principal.

1. **Dónde:** la flecha marca la línea que falló.
2. **Qué tipo** de error es.
3. **Por qué:** la explicación, en la última línea.



Los tres errores más comunes

NameError

```
print(Hola)
NameError: name 'Hola'
is not defined
```

Faltan las comillas: Python cree que **Hola** es una variable.

```
print("Hola")
```

SyntaxError

```
print("Hola"
SyntaxError: '(' was
never closed
```

Algo está mal escrito: aquí falta cerrar el paréntesis.

```
print("Hola")
```

TypeError

```
edad = "20"
edad + 1
TypeError: can only
concatenate str (not "int")
to str
```

No se puede sumar texto con número.

```
int(edad) + 1
```

Antes de arreglar: **lee la última línea y adivina la causa.** El recuadro verde es la versión corregida.

Correo: uniic_bog@unal.edu.co
Teléfono: 3165000 **ext** 12301

UIFCE
UNIDAD DE INFORMÁTICA

6

¡A practicar!

Ejercicios en tres niveles

Correo: uniic_bog@unal.edu.co
Teléfono: 3165000 **ext** 12301

Ejercicios

Avanza hasta donde llegues

Básico — TODOS

1. Crea variables con tu nombre, tu carrera y tu semestre, e imprímelas en una frase.
2. Calcula cuántos minutos tiene una semana.

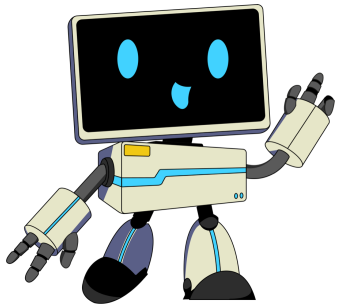
Intermedio

3. Una compra tiene 3 productos con precios distintos. Calcula el total y el total con IVA del 19 %.
4. Usa `type()` para verificar el tipo de cada variable que creaste.

Reto

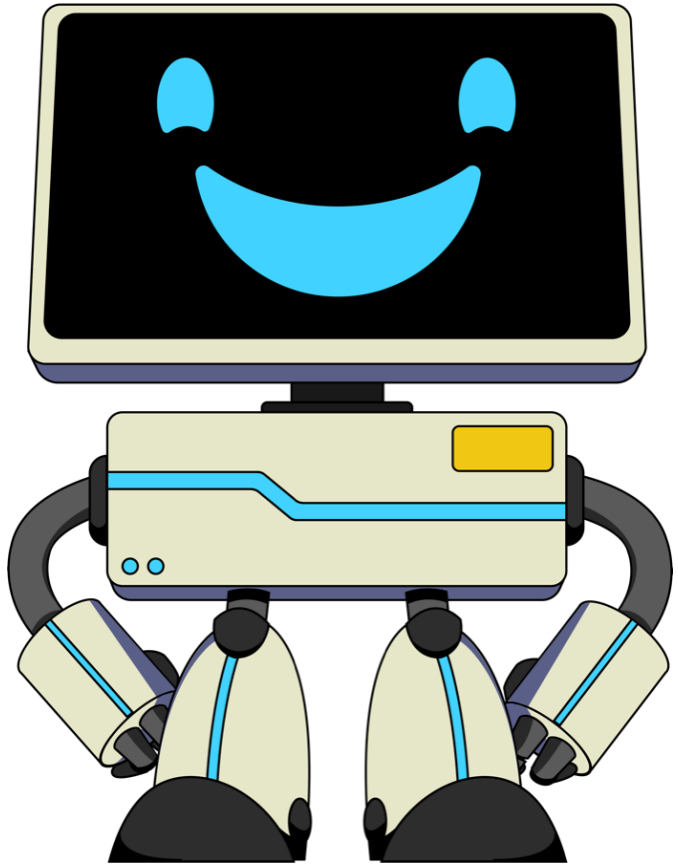
5. Convierte grados Celsius a Fahrenheit: `F = C * 9/5 + 32`. Muestra el resultado con un mensaje claro.
6. Usa `input()` para pedir el nombre del usuario y saludarlo. ¿Qué tipo de dato devuelve?

¿Terminaste? Ayuda a tu pareja o pasa al siguiente nivel.



Tres ideas clave de hoy

1. Un programa es una lista de instrucciones que **se ejecuta en orden**.
2. Las variables **guardan datos**, y cada dato **tiene un tipo**.
3. Los errores son pistas: **leerlos es parte de programar**.



Tarea corta

Terminar los ejercicios básicos e intermedios que no alcanzaron.

Nivelación

Notebook resuelto paso a paso con todos los ejemplos de hoy.

Correo: uniic_bog@unal.edu.co
Teléfono: 3165000 **ext** 12301

UIFCE
UNIDAD DE INFORMÁTICA

¡Gracias!

Próxima sesión: **Condicionales**. ¿Y si el programa pudiera tomar decisiones?

Correo: uniic_bog@unal.edu.co

Tel: 3165000 ***Ext:*** 12301

